

Bonsai Accelerator Hardware Blueprint

Architecture proposal before implementation - Oriol Pont, July 2026

Scope and Benchmark Basis

This document is the pre-implementation hardware blueprint for the Bonsai accelerator. It describes the architecture, submodules, signal groups, and control FSMs that guide development. It fixes service boundaries, data roles, ownership, completion semantics, and evaluation rules. Low-level constants such as register offsets, FIFO depth, exact fixed-point widths, number of compute lanes, arbitration details, and final bus widths remain open for selection during RTL implementation and synthesis.

The software target is Bonsai-1.7B Q1_0, a compact LLM, as previously discussed. The model image is around 242 MB. The course board is a Sipeed Tang Nano 9K with a Gowin LittleBee GW1NR-9 FPGA running a NEORV32 System-on-Chip (SoC), far too restricted to hold the full model image.

The three-tier benchmark exploration in `docs/01-bonsai-bottleneck-benchmark.pdf` has allowed defining the scope. To recap, Tier 1 uses `llama.cpp` as the full software reference and shows two important runtime regimes: short-context inference is dominated by Q1_0 matrix operations, while long-context inference shifts toward attention and KV-cache work. Tier 2 exposes the same Bonsai path in a custom, self-contained C++ runner and measures repeated Q1_0 matrix-vector calls over 128-weight packed groups, plus per-layer attention over growing KV history. Tier 3 supplies simulated NEORV32 software-cycle baselines over board-sized and Bonsai-inspired operation shapes.

Tier 2 remains the full-model accounting layer used for extrapolation. Tier 3 defines the SoC-facing operation boundaries and pre-acceleration cycle references. The hardware engines will preserve those boundaries so that later results can combine accelerator measurements with Tier 2 call counts.

The final architecture therefore contains two proposals, each of which addresses one of the two bottlenecks identified:

1. **Proposal A: Q1_0 by Q8_0 matrix-vector engine.** accelerate the packed fixed-weight dot-product backend used by Bonsai linear layers.
2. **Proposal B: streaming attention/KV engine.** execute K/V append, QK scoring, stable softmax normalization, and weighted-V accumulation while improving long-context data delivery through FIFO and memory streaming.

Common Top-Level Architecture

In the proposed architecture, the accelerator is a co-processor to the NEORV32 CPU inside the SoC. The CPU is the system controller, running the Bonsai-like harness that orchestrates model execution and launches accelerator work at the target service boundaries. The accelerator provides two services: a Q1_0 by Q8_0 matrix-vector service and a streaming attention/KV service.

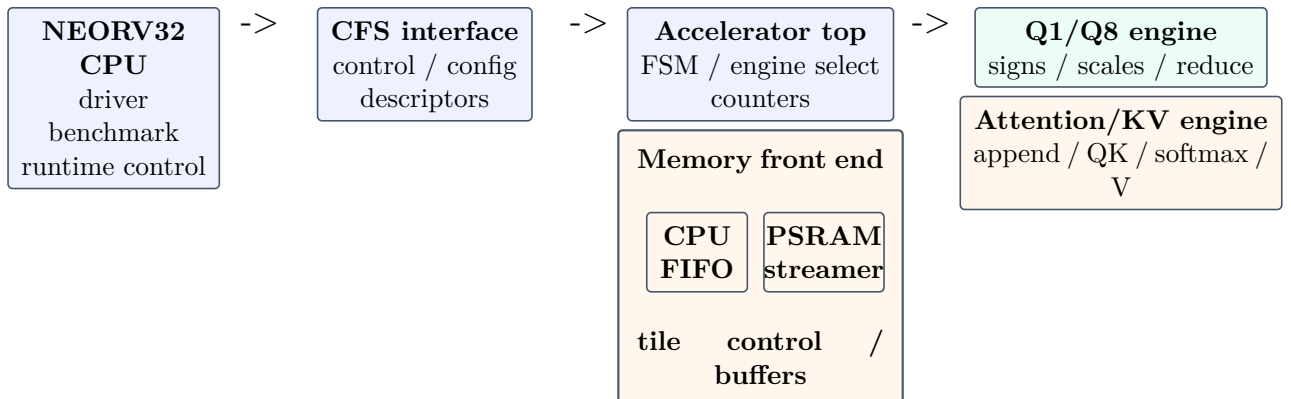


Figure 1: Common architecture used by both proposals. CFS carries commands and counters to `accel_top`, which selects the engine and data path. Memory management selects `CPU_PUSH`, where software provides FIFO payloads, or `MEM_STREAM`, where the PSRAM streamer moves descriptor-addressed tiles. Both modes share the tile buffers and engines, isolating data-delivery gains from compute-engine gains.

Top-Level Modules and Signal Groups

This section fixes the architectural contract shared by both engines: CPU-visible command and status state, one of two data-delivery modes, role-tagged tile staging, engine dispatch, completion, and counters. Register encoding and cycle-level wiring remain RTL decisions.

Top-Level Submodules

Submodule	Responsibility	Architectural state
cfs_reg_file	CPU-visible CFS command, descriptor, request-status, result-status, FIFO, and counter access point.	command lifecycle, descriptors, request status
frontend_control	Translates engine tile demand into CPU_PUSH FIFO service or MEM_STREAM memory traffic and coordinates tile placement.	active role/tile, direction, transfer mode
stream_frontend	CPU_PUSH tagged ingress and egress queues. Couples CPU-provided or CPU-consumed words to the active frontend request.	FIFO occupancy, active tag
memory_streamer	MEM_STREAM descriptor-driven reads and writes over an abstract burst-memory interface.	burst progress, descriptor position
local_buffer_bank	Shared FPGA-local storage for role-tagged tiles consumed or produced by an engine.	tile validity, role/tile ownership, logical length
accel_top	Validates a command, selects one committed service engine, coordinates frontend and engine lifecycles, and publishes terminal status.	service mode, transfer mode, command state
q1_matvec_engine	Q1_0-by-Q8_0 matrix-vector service endpoint.	row/group/chunk progress
attn_kv_engine	Attention and KV append service endpoint.	head/context/phase progress
counter_block	Captures command cycles, engine-active cycles, frontend waits, traffic, and completed work units.	cycle and event counters

The shell has three stable boundaries. CFS carries commands, descriptors, FIFO service, request visibility, status, and counters. The frontend presents one role-tagged tile contract regardless of transfer mode. The engine boundary presents a common lifecycle and tile-demand contract to exactly one selected engine.

Each command selects one committed service mode, Q1_MATVEC or ATTN_KV, and one transfer mode, CPU_PUSH or MEM_STREAM. Its descriptor set has one entry per semantic role. Q1_MATVEC uses Q8_INPUT, Q1_WEIGHTS, and OUTPUT. ATTN_KV uses QUERY, CURRENT_K, CURRENT_V, K_CACHE, V_CACHE, and OUTPUT; implementations that externalize score tiles may add SCORES. Every entry identifies the role and logical extent; MEM_STREAM entries also provide the addressing and stride information needed to locate tiles.

A tile transaction is identified by semantic role, tile index, direction, and logical length. In CPU_PUSH, the frontend publishes the current unsatisfied transaction through CFS-visible request status. Software reads request-valid plus the requested role, tile index, direction, and remaining length, then supplies matching tagged input FIFO data or drains matching tagged output FIFO data. Acceptance advances or clears that request. In MEM_STREAM, the same transaction selects a descriptor entry and becomes burst-memory traffic. Packet boundaries, register sequencing, and detailed backpressure behavior are defined by RTL while preserving this observable contract.

Top-Level Signal Groups

Interface group	Direction	Representative concepts	Architectural contract
Clock and reset	global input	clock, reset	Shared timing and reset domain for the accelerator shell.
CFS access	CPU <-> CFS	request/response, address, data	CPU access to command, descriptors, request status, tagged FIFOs, result status, and coun-

Interface group	Direction	Representative concepts	Architectural contract
			ters. Register protocol and address map are deferred to RTL.
Command and descriptors	CFS -> top/frontend	start/clear, service mode, transfer mode, role descriptors	A validated command selects Q1_MATVEC or ATTN_KV, selects CPU_PUSH or MEM_STREAM, and supplies the role-based shape and location metadata.
Lifecycle and counters	top/counters -> CFS	busy/done/error, selected service, event snapshots	CPU-readable command state, terminal outcome, service identity, and performance observations.
CPU_PUSH status	frontend -> CFS/CPU	valid, role, tile, direction, remaining length	Identifies the exact tile service currently requested from software. Status remains stable until matching FIFO progress occurs or the command terminates.
Tagged FIFO vice	CFS/CPU <-> stream frontend	valid/ready, data, role/tile tag, occupancy	Carries CPU-supplied input and CPU-consumed output for the published request. Packet-boundary mechanics and register operations are deferred to RTL.
Abstract memory	burst memory streamer <-> memory controller	read/write request, address/length, data handshake, completion/error	MEM_STREAM converts role descriptors and tile indices into bursts. Exact controller channel pins and physical PSRAM signaling are deferred to RTL.
Role-tagged tile	frontend/buffer <-> engine	request/accept, role, tile, direction, logical length, data availability	Transfers semantic tiles through shared local storage. Bank selection, ports, pointers, and arbitration are deferred to RTL.
Engine lifecycle	top <-> selected engine	launch/accept, service identity, busy/done/error	Exactly one engine owns a command from accepted launch through terminal completion.
Frontend stones	mile- frontend -> top	first tile ready, transfer complete, error	Provides the milestones required for launch ordering and final command completion while leaving scheduling details to RTL.

These groups preserve one engine-facing contract across CPU_PUSH and MEM_STREAM, including identical semantic roles and tile indices. The following section defines the control Finite-State Machines (FSMs) that coordinate command, frontend, buffer, and engine progress.

The two transfer modes form an implementation progression. CPU_PUSH is the straightforward first hardware integration: compute moves into the selected engine, while software still relays every payload through the CFS FIFOs. MEM_STREAM is the target memory architecture: the frontend uses descriptors, PSRAM bursts, and local tile buffering to move the same payloads directly and reduce CPU involvement and engine delivery stalls.

Control Finite-State Machines

accel_top Finite-State Machine

accel_top is the main top-level controller. It validates commands, starts the selected data path, launches the selected engine after the first tile is ready, and reports completion after the final transfer finishes.

State	Action / outputs	Condition	Next state
IDLE	Clear transient control on reset; hold published status and result metadata until acknowledged; accept and latch a valid command when the prior result is clear.	valid command or reset/acknowledge	PREPARE or stay
PREPARE	Set <code>status_busy</code> , validate the latched descriptor and resources, select and start the CPU-push or memory frontend, and prime the first required tile.	first tile ready or fault	RUN or ERROR
RUN	Launch the selected engine once, then run engine execution concurrently with frontend bank refill and output drain while counting active and stall events.	engine complete or fault	DRAIN or ERROR
DRAIN	Keep the command busy until the final CPU-push FIFO output is drained or the final PSRAM output/KV writeback response is accepted; then snapshot counters and publish result metadata.	final transfer complete or fault	DONE or ERROR
DONE	Set <code>status_done</code> , clear <code>status_busy</code> , and retain status and result metadata until software acknowledges.	software acknowledges or waiting	IDLE or stay
ERROR	Set <code>status_error</code> , clear <code>status_busy</code> , latch a generic fault code, and hold command state until software clears the error.	software clears error or waiting	IDLE or stay

In `CPU_PUSH`, software services FIFO data and level/status registers while `status_busy` = 1: it supplies requested input packets and drains output packets as they become available. `status_done` follows the final output drain. In `MEM_STREAM`, completion follows the final accepted memory write response. These rules give both modes the same command boundary and avoid dependence on a drain-after-done driver sequence.

stream_frontend Channel Control

The CPU-push frontend has independent ingress and egress channel control, allowing input refill and output drain during the same engine interval:

Channel	Lifecycle	Responsibility
Ingress	IDLE -> RECEIVE -> COMPLETE	Accept role- and tile-tagged CPU words, apply backpressure when the assigned bank is unavailable, and publish packet completion or error.
Egress	IDLE -> SEND -> COMPLETE	Expose ready output packets through FIFO level/data registers, retain words during CPU backpressure, and publish packet completion or error.

Either channel can return to `IDLE` for the next tile while the other remains active. Channel faults propagate to `accel_top` through `frontend_error`.

memory_streamer FSM

The memory streamer is the optimized data-delivery path. `frontend_control` translates role- and tile-tagged requests into descriptor-derived transfers; the streamer executes each PSRAM read or write transaction, fills local tile buffers, and writes append/output payloads back to memory.

State	Action / outputs	Condition	Next state
IDLE	Wait for a <code>MEM_STREAM</code> transaction and latch its direction, address, length, role, tile tag, and bank as metadata.	transaction accepted	REQUEST
REQUEST	Present the metadata-derived read or write request to the external-memory interface while retaining it across back-pressure.	request accepted or waiting or fault	TRANSFER or stay or ERROR
TRANSFER	For a read, place accepted response words into the selected bank; for a write, send bank words and accept the final response. Count transferred bytes, with delayed progress represented by a self-loop and wait-counter event.	transaction complete or waiting or fault	COMPLETE or stay or ERROR
COMPLETE	Publish transaction completion and the final byte count to <code>frontend_control</code> .	frontend acknowledges	IDLE
ERROR	Latch a generic transaction fault and notify <code>frontend_control</code> .	error cleared	IDLE

local_buffer_bank Bank-State Control

The local buffers are a shared staging area. They decouple CPU/PSRAM transfer timing from engine timing and give both engines a common tiled data source and sink. Each bank tracks its own state, owner, tile index, valid length, and read/write pointers:

Bank state	Meaning	Next state
EMPTY	Available for assignment to the selected frontend or engine.	FILLING
FILLING	The assigned producer writes one input or output tile and advances the write pointer.	READY or error
READY	The complete tile is valid and available to its assigned consumer.	IN_USE
IN_USE	The engine consumes an input tile, or the selected frontend drains an output tile.	EMPTY or error

Because the banks advance independently, one bank can be **FILLING** while another is **IN_USE**. The frontend reports first-tile readiness, final transfer completion, and transfer errors to `accel_top`; detailed arbitration and the number and depth of banks remain RTL design parameters.

Counter Policy

The measurement contract exposes command and engine elapsed cycles, engine-active cycles, input, output, and frontend wait cycles, logical work, and physical frontend bytes; useful phase counters may be selected for Tier 3 comparisons. Each engine cycle is classified once as active, input wait, output wait, or control overhead, making utilization equal to active cycles divided by engine elapsed cycles. Frontend activity and waits are reported independently because tile transfer can overlap engine execution. Driver-observed service time spans command issue through final result delivery, while CPU fill, polling, drain, and blocked intervals remain separate.

This completes the top-level SoC contract and control interface. The next sections specialize the Q1_0 by Q8_0 engine and the streaming attention/KV engine.

Proposal A: q1_matvec_engine

Proposal A is the compute-first tensor accelerator. Its target primitive is the Q1_0 by Q8_0 matrix-vector service measured in Tier 3. Decode applies this fixed-weight operation for each current token, while prefill repeats the same linear transforms over prompt positions or batches.

The engine boundary receives prequantized activations and packed Q1_0 records. One Q1_0 group contains 128 sign bits and one weight scale. One Q8_0 block contains 32 signed eight-bit activation values and one activation scale, so each Q1_0 group consumes four Q8_0 blocks. For each block, the hardware performs sign-controlled addition/subtraction, combines the weight and activation scales with the integer partial sum, accumulates into a wide row accumulator, and finally saturates the output to signed 16-bit form. An ingress format adapter converts GGUF FP16 weight scales into the selected internal fixed-point representation, matching the conversion included by the Tier 3 fixture path.

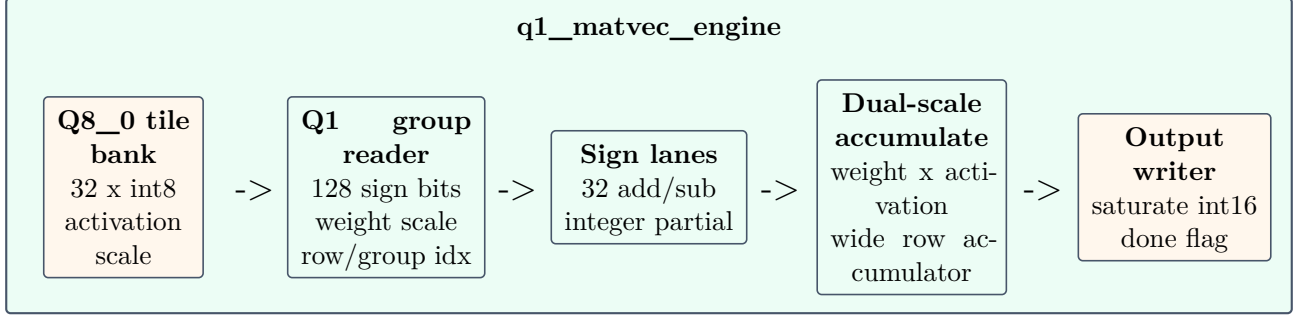


Figure 2: Zoom-in block diagram for q1_matvec_engine.

Q1_0 by Q8_0 Matvec FSM

This FSM is local to q1_matvec_engine. Activation quantization occurs before the service boundary, matching the Tier 3 baseline. The engine consumes packed Q1_0 and Q8_0 records through the shared tile interface. A Q8 activation vector is logically reusable for every output row. Whether its blocks remain resident or are refetched depends on tile-buffer capacity; the physical-byte counters measure the implemented choice separately from logical Q8 block uses.

State	Action / outputs	Condition	Next state
IDLE	Keep <code>engine_busy = 0</code> ; wait for the top-level launch handshake.	<code>engine_start</code> <code>engine_kind</code> <code>q1_matvec</code>	with = PREPARE
PREPARE	Latch dimensions, Q1/Q8 counts, buffer selectors, scale format, and output format; request or select the prequantized Q8_0 activation vector. Hold descriptor and request state during input backpressure.	descriptor and activation ready or still waiting or descriptor/buffer fault	INIT_ROW or PREPARE or ERROR
INIT_ROW	Clear the wide row accumulator and initialize the Q1-group index and four-block index for the current row.	row initialized	PROCESS_GROUP
PROCESS_GROUP	Process exactly one Q1_0 group: obtain its 128 sign bits and weight scale, consume the four corresponding Q8_0 blocks of 32 values, perform sign-controlled add/sub for each block, combine each integer partial with the Q1 and Q8 scales, and accumulate all four scaled partials into the row accumulator. Hold row, group, block, and partial state when an input is unavailable. After block four, advance to the next group; after the final group, complete the row.	waiting or more groups remain or final group for this row or protocol fault	PROCESS_GROUP or WRITE_ROW or ERROR

State	Action / outputs	Condition	Next state
WRITE_ROW	Saturate the completed row accumulator to signed 16-bit form and present it to the shared output bank. Hold the row result during output backpressure; after acceptance, advance the row counter.	output blocked or output accepted with more rows or final output accepted or protocol fault	WRITE_ROW or INIT_ROW or DONE or ERROR
DONE	Assert <code>engine_done</code> after the final row enters the shared output bank, clear <code>engine_busy</code> , and expose engine-local counters.	top acknowledges	IDLE
ERROR	Assert <code>engine_error</code> , clear <code>engine_busy</code> , and latch the local failure reason.	top clears command	IDLE

Engine completion marks completion of Q1_0 by Q8_0 computation and local result production. Command completion follows after `accel_top` observes final FIFO drain or PSRAM writeback through the shared frontend.

Proposal B: attn_kv_engine

Proposal B pairs an attention compute engine with the shared memory frontend. Long-context cost grows as every decoded token appends new K/V data and traverses the stored history. The engine owns the attention phase order and tile-level computation, while the frontend supplies cache tiles and commits append and output tiles through the selected data-delivery path.

The reference service receives backend-ready Q, current K, and current V. Q and K have already passed their head normalization and RoPE preparation. The service appends current K/V at the decode position, streams K over the context window to compute scaled QK scores, applies stable softmax, streams V, accumulates the weighted output, and writes the signed-16 attention result. This normalization procedure is the validation mode corresponding to Tier 3.

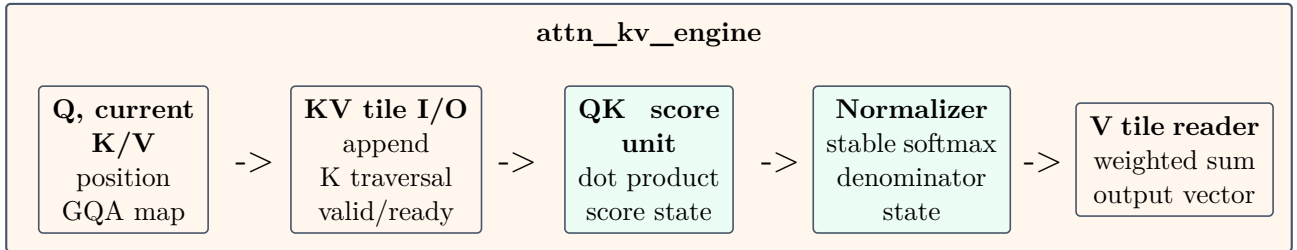


Figure 3: Zoom-in block diagram for `attn_kv_engine`.

Attention/KV Engine FSM

It shares the common engine launch/completion contract. Its semantic phases preserve the measured Tier 3 order: K/V append first, followed for each query head by Q loading and head preparation, K traversal, stable normalization, V traversal, and output writeback.

State	Action / outputs	Condition	Next state
IDLE	Keep <code>engine_busy</code> = 0; wait for the top-level launch handshake.	<code>engine_start</code> with <code>engine_kind</code> = <code>attn_kv</code>	PREPARE
PREPARE	Latch head counts, head dimension, context length, append position, tile-buffer selectors, normalization mode, and output format. Hold descriptor state during backpressure.	descriptor accepted or still waiting or descriptor invalid	APPEND or PREPARE or ERROR

State	Action / outputs	Condition	Next state
APPEND	Present the current token’s K and V as an append tile, retain both in the local current-position cache view, and count logical write bytes. Hold append state until the tile interface accepts the data.	append accepted or append delayed or protocol fault	PREPARE_HEAD or APPEND or ERROR
PREPARE_HEAD	Load or select Q for the current query head, map that head to its KV head, and clear per-head score, normalization, output, and context state. Hold the selected head while Q is unavailable.	head ready or Q unavailable or buffer fault	SCORE or PREPARE_HEAD or ERROR
SCORE	Traverse K across the full context, including the local current-K view at the append position; compute scaled QK scores in context order, store every Tier 3 reference score locally, and count logical read bytes. Hold the context index and partial score when a tile is delayed.	all scores stored or tile delayed or tile fault	NORMALIZE or SCORE or ERROR
NORMALIZE	Apply stable softmax to the stored scores in Tier 3 order: maximum subtraction, exponent evaluation and accumulation, then denominator division. Hold normalization state while an arithmetic unit is busy.	normalization complete or arithmetic busy or numeric/protocol fault	VALUE or NORMALIZE or ERROR
VALUE	Traverse V in the same context order, including the local current-V view at the append position; pair each vector with its normalized score, accumulate the weighted output, and count logical read bytes. Hold context and accumulator state when a tile is delayed.	weighted output complete or tile delayed or tile fault	WRITE or VALUE or ERROR
WRITE	Convert the completed attention output to signed-16 form and present it to the shared output bank. Hold the result during output backpressure; after acceptance, advance the query-head counter.	output blocked or output accepted with more heads or final output accepted or protocol fault	WRITE or PREPARE_HEAD or DONE or ERROR
DONE	Assert <code>engine_done</code> after the final attention result enters the shared output bank, clear <code>engine_busy</code> , and expose attention/KV counters.	top acknowledges	IDLE
ERROR	Assert <code>engine_error</code> , clear <code>engine_busy</code> , and latch the local failure reason.	top clears command	IDLE

The local cache view makes the newly appended K/V available when the current context position is scanned. The shared frontend can complete the corresponding FIFO drain or PSRAM writeback in parallel, and `accel_top` holds command completion until that transfer finishes.

The Tier 3 compatibility implementation keeps every reference score in FPGA-local storage. Larger contexts require scalable score storage or normalization and separate validation against this reference. The FSM states express per-head semantic order; a GQA-optimized tile schedule may interleave query heads mapped to one KV head so they consume a retained K/V tile while preserving each head’s ordered operations and results. Logical K/V work remains equal to Tier 3, while physical frontend bytes reveal the resulting reuse.

Evaluation Contract

The hardware benchmark preserves the Tier 3 operation dimensions, input formats, phase boundaries, and output checks for the compatibility profiles while adding hardware-owned elapsed, active, wait, work, and traffic measurements. The first RTL version uses `CPU_PUSH` as the straightforward hardware baseline. The optimized version uses `MEM_STREAM` with the same compute engine, allowing memory-path gains to be isolated.

Memory-path comparisons use the same prepared payload image and declared result destination. Fixture generation and initial placement occur before timing. In `CPU_PUSH`, the driver relays payloads between the declared backing memory and the CFS FIFOs, including final result placement. In `MEM_STREAM`, the descriptor-driven frontend performs those transfers directly. The timed service boundary includes the complete relay or stream path through result placement, so the comparison measures the removal of CPU-mediated data movement around an unchanged engine workload.

Compatibility Profiles

The Tier 3 compatibility profiles establish the first correctness and software-cycle comparisons:

Engine	Board profile	Cycles	Bonsai profile	Cycles
Q1 by Q8	1 x 128 synthetic	7,934	1 x 2048, GGUF row	195,602
Attention/KV	H=1, KVH=1, D=32, C=2	489,007	H=2, KVH=1, D=16, C=2	494,741

These profiles define the implementation acceptance scope:

- Proposal A validates one complete row at the 128-element board work-unit and 2,048-element Bonsai hidden width, including complete command timing and output comparison.
- Proposal B validates the declared `ctx = 2` board and GQA services under `CPU_PUSH` and `MEM_STREAM`, preserving the engine workload while exposing data-delivery effects.

Larger row counts and context sweeps extend this characterization toward sustained throughput. They remain future work because they require additional fixture storage and substantially longer cycle simulation while leaving the service contracts unchanged.

Summary

This blueprint defines a shared accelerator shell with two fixed service contracts and two selectable data-delivery paths. The Q1_0 by Q8_0 engine matches the packed matvec operation measured in Tier 3. The attention/KV engine matches the append, score, stable-softmax, and weighted-value service. `CPU_PUSH` establishes a straightforward hardware baseline, while `MEM_STREAM` uses PSRAM descriptors and local ping-pong tiles to reduce data-delivery stalls around the same compute engines, primarily targeting Proposal B.

Implementation follows this blueprint one service at a time: validate each Tier 3 compatibility profile, compare the two attention data paths around the same engine, and synthesize proposal-specific SoCs for board feasibility.